

To calculate the **F1 score** for this QA example, we treat it like a **token-level comparison** between:

- **Prediction (model response)**

"Berlin is the capital of Germany."

- **Ground truth**

"Berlin"

---

## ✓ Step 1: Normalize both texts

Typical QA evaluation (like SQuAD) applies normalization:

- Lowercase
- Remove punctuation
- Remove articles ( a , an , the )
- Trim extra whitespace

### After normalization:

**Prediction:**

berlin is capital of germany

**Ground truth:**

berlin

---

## ✓ Step 2: Tokenize

**Prediction tokens:**

["berlin", "is", "capital", "of", "germany"]

→ 5 tokens

**Ground truth tokens:**

["berlin"]

→ 1 token

---

## ✓ Step 3: Count overlapping tokens

Common tokens = ["berlin"]

Number of overlapping tokens = 1

---

## ✓ Step 4: Calculate Precision and Recall

### Precision

$\text{Precision} = \frac{\text{\# overlapping tokens}}{\text{\# prediction tokens}} = \frac{1}{5} = 0.2$

### Recall

$\text{Recall} = \frac{\text{\# overlapping tokens}}{\text{\# ground truth tokens}} = \frac{1}{1} = 1.0$

---

## ✓ Step 5: Calculate F1 Score

$$F1 = \frac{2 \times (\textit{Precision} \times \textit{Recall})}{\textit{Precision} + \textit{Recall}}$$

$$F1 = \frac{2 \times (0.2 \times 1.0)}{0.2 + 1.0}$$

$$F1 = \frac{0.4}{1.2} = 0.3333$$

---

## ✓ ✓ Final F1 Score = 0.33

---

## ✓ Why it's not 1.0

Even though the answer is correct, F1 penalizes extra words.

The model returned a **full sentence**, but the ground truth is just "**Berlin**", so precision is low.

---

## ✓ If prediction was just:

"Berlin"

Then:

- Precision = 1
  - Recall = 1
  - F1 = **1.0**
- 

If you'd like, I can also give you a small Python function to compute QA-style F1 automatically.